

Operating Systems:

OS is necessary for resource management that means allocation of CPU, GPU, memory and disk. OS is necessary for not making apps bulky and not violating DRY Principle. That is achieved by abstraction.

OS acts as an interface between the user and the computer hardware.

OS has access to computer hardware.

OS does execute the programs by providing isolation and protection.

Goals of OS:

Maximum CPU utilisation.

No process starvation.

High priority execution.

Types of OS:

Single Process: Processes 1 task at once. Violates point 1, 2, 3. Example: MS DOS

Batch Processing: Similar jobs are batched together. Individual jobs inside batches get processed. Violates all 3.

Multi-programming OS: Single CPU and Queue of jobs. Works on context switching using process control blocks.

Multi-Tasking OS: Single CPU + context switching + time sharing. Max CPU utilisation.

Multi-processing OS: Multiple CPU + context switching + time sharing.

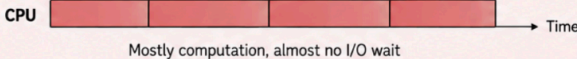
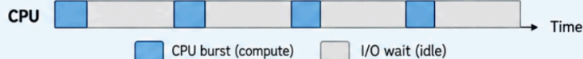
Distributed OS: Loosely coupled multiple machines. Interface for multiple computers connected together. Fault tolerant and provides parallelism.

RTOS: Real time error free execution.

Process:

Program (code) under execution. Program goes from disk to RAM before getting executed.

Type of processes in OS:

CPU-bound vs I/O-bound Processes	
Difference, Examples, and Suitable Operating Systems	
 CPU-bound Process Spends most of the time doing computations.	 I/O-bound Process Spends most of the time waiting for input/output.
Definition Requires more CPU time doing processor computations than waiting for input/output. Keeps the CPU busy with long bursts of calculations.	Definition Spends more time waiting for input/output operations (like keyboard, disk, or network reads) than executing code on the processor. Features short bursts of CPU usage.
Execution Behavior - Long CPU bursts - Very little I/O wait time - CPU is the bottleneck	Execution Behavior - Short CPU bursts - Long I/O wait time - I/O devices are the bottleneck
Examples - Scientific calculations (e.g., simulations) - Video rendering / encoding - Machine learning training - Data compression (e.g., zip) - Compiling large codebases	Examples - File reading/writing (disk I/O) - Network communication (web servers, APIs) - Database queries - User input (keyboard, mouse) - Media playback (audio/video streaming) - Printing, sensor data collection
Execution Timeline (CPU-bound)  Mostly computation, almost no I/O wait	Execution Timeline (I/O-bound)  Legend: CPU burst (compute) (blue), I/O wait (idle) (grey)
Best Suitable OS - Batch Operating Systems (e.g., for large computations) - Real-Time Operating Systems (for time-critical computation tasks) - General-purpose OS (e.g., Linux, Unix) also perform well  Designed to maximize CPU utilization and throughput.	Best Suitable OS - Time-sharing / Interactive Operating Systems (e.g., Windows, Linux, macOS) - Network Operating Systems (for handling many I/O operations) - Distributed Operating Systems (for network-heavy workloads)  Designed to handle multiple I/O-bound processes efficiently (using techniques like multiprogramming, async I/O, etc.).
Key Takeaways CPU-bound → computation heavy → use systems that maximize CPU usage (Batch, Real-Time, General-purpose). I/O-bound → I/O wait heavy → use systems that handle many I/O operations and provide good responsiveness (Time-sharing, Network, Distributed). - Most modern OS (like Linux/Windows) handle both using scheduling algorithms, but their design focus may vary based on the use case.	

in case of CPU bound - multi programming helps (parallelism)

in case of IO bound - context switching helps (concurrency)

Thread:

Light weight processes that can be executed independently, but uses shared memory with a process.

Each thread has its own TCB(thread control block) for context switching.

Memory in threads is as follows:

stack1, stack2 stack3 heap, data, text.

Each thread has its own STACK.

Efficient for multi processing environments so that program can be broken down into independent threads and then get executed in parallel. For single CPU it will take the same time and switch contexts only.

Multi Tasking	Multi Threading
More than 1 process	More than 1 thread
Isolation of memory and protection	No isolation of memory and protection
Processes are scheduled	Threads are being scheduled.
1 CPU	More than 1 preferred.
Process Context Switching	Thread Context Switching
Slow switching	Fast switching
Includes switching of memory and address.	Does not switches memory and address.
CPU cache state is flushed.	CPU cache state is preserved.

Benefits of Multi threading:

1. Responsiveness: Useful in interactive apps, can be easily threaded - if 1 thread stops, 2nd can still take input.
2. Resource Sharing: Threads have shared memory, so no IPC required.
3. Economy: less overhead in context switching than a process.
4. Multi Core Utilisation: So more scalable.

Components of OS:

User space interacts with kernel and kernel with OS.

1. User space - GUI and CLI
2. Kernel - First part to load

User Space:

Does not have access to the hardware.

Applications interact only with user space as we have to keep security.

Kernel:

Heart of OS, directly interacts with OS.

has all access and works on system calls.

Shell: (.sh)

Command interpreter which reads commands from the user and gets them executed.

Terminal:

Window for the shell to work in.

How do processes interact?

IPC - inter process communication (for communication b/w 2 independent processes.)

example - clipboard (copy from one app, paste in another).

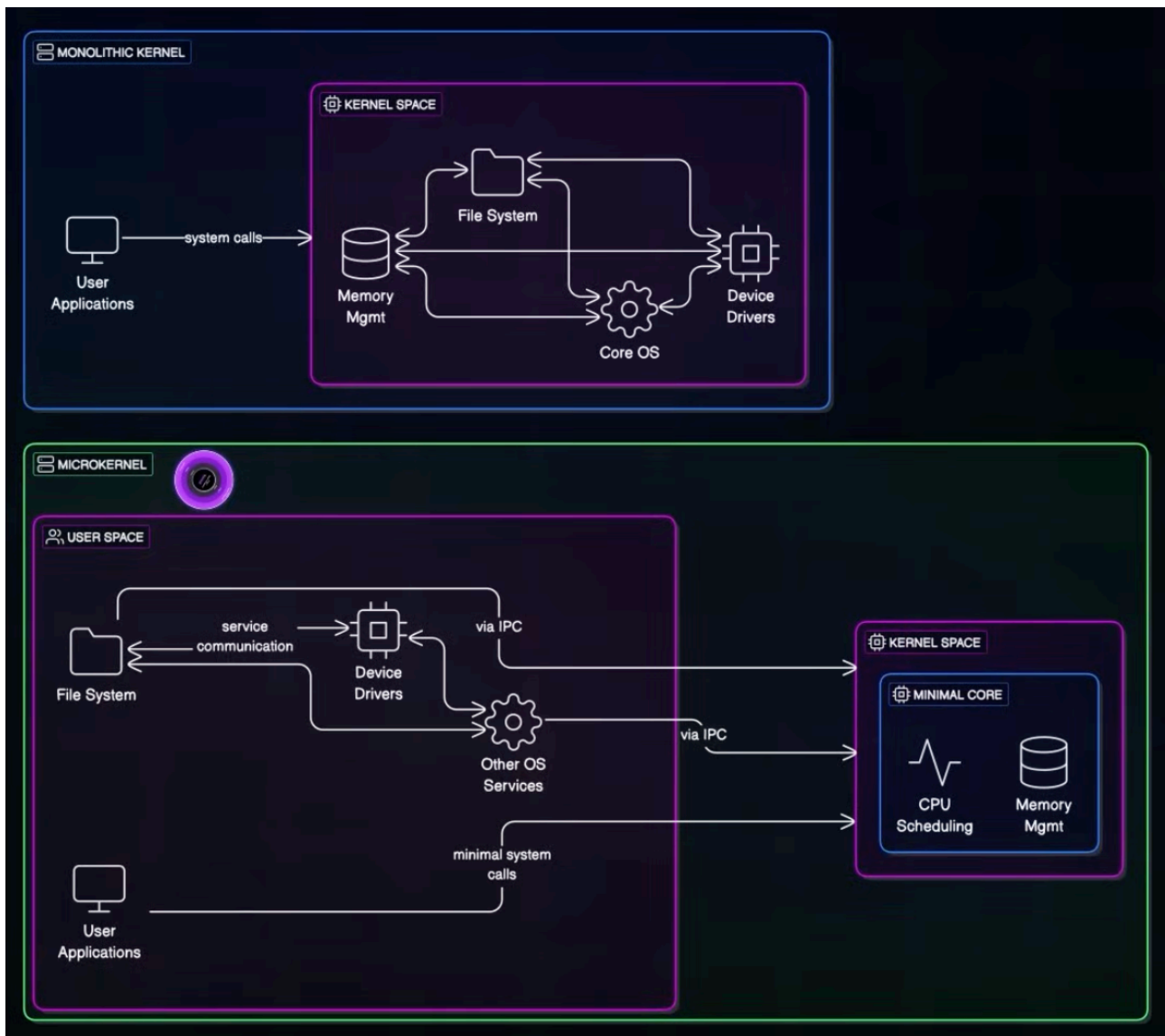
2 methods for IPC are Shared Memory and Message Passing.

Functions of Kernel:

1. Process Management - Creation, termination, scheduling, Synchronisation, Communication
2. Memory Management - Allocate / Deallocate, keeping track of allocated memory.
3. File Management - Create, delete, directory management
4. I/O Management - Input and output devices, pen drives etc (Spooling, Buffering, Caching)

Types of Kernel:

1. Monolith Kernel: Everything in kernel. switch b/w kernel and user space using software interruption. it is fast.
2. Micro Kernel: I/O and file managed in user space and rest in kernel. more robust.
3. Hybrid Kernel: File manage in user space and rest in kernel. Speed and design of mono and robust from micro.
4. Nano/Exo Kernel: Beyond Scope.

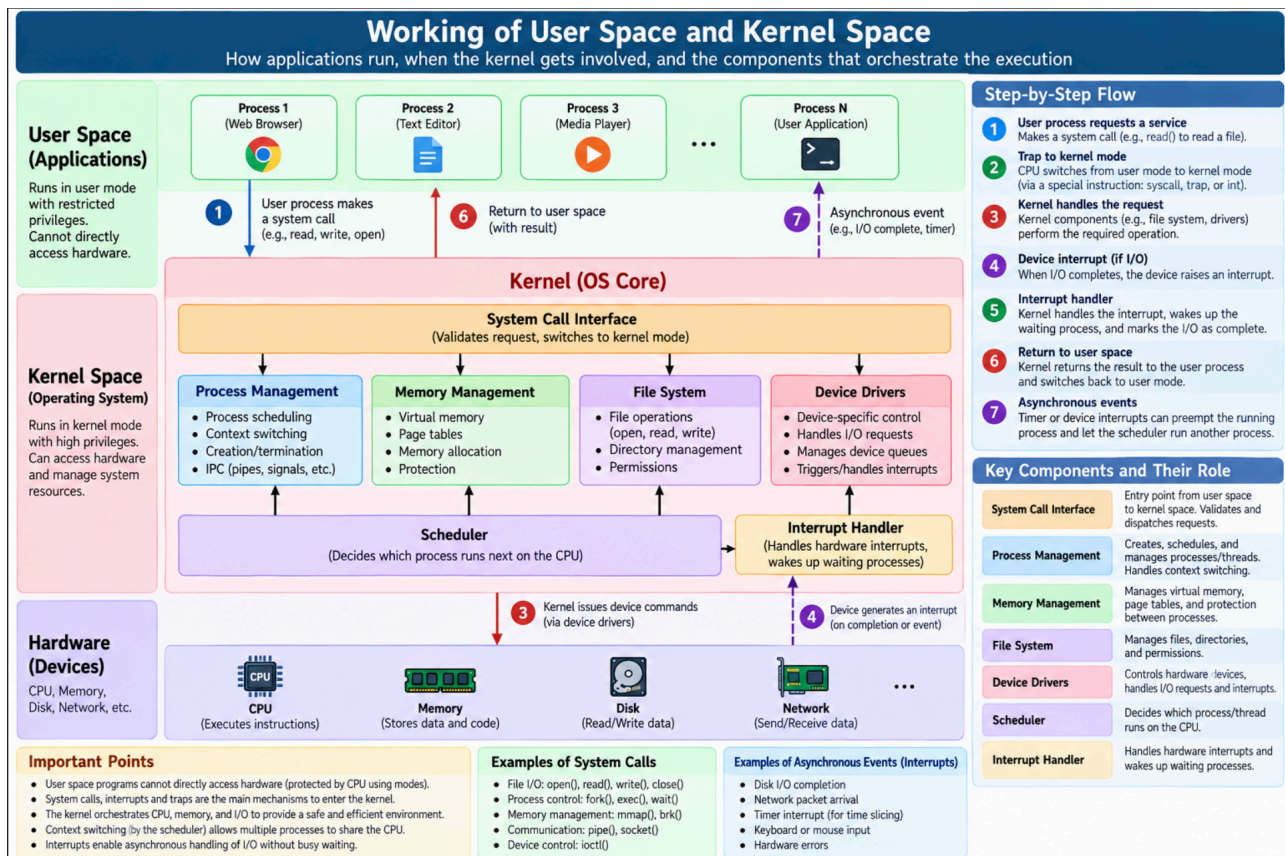


System Calls in Operating System:

A system call is a mechanism using which a user program can request a service from the kernel for which it does not have the permission to perform. They are implemented in C.

Flow of how it works:

User executes a process. (User space)
 Gets system call using the **"Glibc"** library. (US)
 Executes system call to create a process. (KS)
 Return to US.



Important Points

- User space programs cannot directly access hardware (protected by CPU using modes).
- System calls, interrupts and traps are the main mechanisms to enter the kernel.
- The kernel orchestrates CPU, memory, and I/O to provide a safe and efficient environment.
- Context switching (by the scheduler) allows multiple processes to share the CPU.
- Interrupts enable asynchronous handling of I/O without busy waiting.

Examples of System Calls

- File I/O: open(), read(), write(), close()
- Process control: fork(), exec(), wait()
- Memory management: mmap(), brk()
- Communication: pipe(), socket()
- Device control: ioctl()

Examples of Asynchronous Events (Interrupts)

- Disk I/O completion
- Network packet arrival
- Timer interrupt (for time slicing)
- Keyboard or mouse input
- Hardware errors

Types of System Calls:

- Process Calls: end, abort, load, execute, get and set attributes, wait event, signal event, allocate/free memory.
- File Management: open, close, delete, create, read, write, search.
- Device Management: request, release, read, write, attach or detach devices.
- Information Maintenance: get and set time and date, get and set system data, get file attributes.
- Communication Management: create and delete connections, send, receive transfer status.

Examples of Windows & Unix System calls:

Category	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle() SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	open () read () write () close () chmod() umask() chown()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Management	GetCurrentProcessID() SetTimer() Sleep()	getpid () alarm () sleep ()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe () shmget () mmap()

Different Storages in Computer:

First 3 are primary memory and rest secondary memory ->

1. Register
2. Cache
3. Main Memory
4. Electronic Disk
5. Magnetic Disk
6. Optical Disk
7. Magnetic Tapes

Comparison:

1. Cost: Register is the costliest (Silicon and Transistors).
2. Access Speed: same order.
3. Storage Size: opposite fashion in this.
4. Volatility: Primary flushes but secondary persists after booting.

How OS creates a process:

Programs are in disk.

Steps:

1. Load the program and static data in the memory (RAM).
2. Allocate runtime stack (stack frame) (part of memory used for local variables, function arguments, return values).
3. Allocate runtime heap (part of memory used for dynamic allocation).
4. I/O task (i/p handle, o/p handle, error handle).
5. OS hands off control to main().

Architecture of Processes:

stack heap, data, text

text is compiled code and data means static and global data.

2 famous errors: stack overflow and out of memory: recursion w/o base case and not freeing memory.

Attributes of Processes:

OS lists processes in process table and each entry in this table is about PCB (Process Control Block).

PCB:

This is a data structure that stores all the data about the process.

Process ID: unique identifier.

Program Counter: stores the address of the next machine instruction that the process should execute.

Process State: Current state (new, waiting, running).

Priority: Simple priority.

Register: Stores the CPU registers so that we can easily know the last state of process.

Open File List: Opened files at current execution.

Open Devices List: IO devices that were open at last execution.

Different Process States:

1. New: program becomes process (PCB initialised).
2. Ready State: is sitting in memory in ready queue. (waiting for CPU)
3. Running: CPU allotted.
4. Waiting: Waiting for IO completion.
5. Termination: Process finished.

New to Ready: Job queue when in new and transferred via job scheduler / long term scheduler. (LTS) to ready queue.

Ready to Running: Ready queue when in ready and transferred via Short term scheduler / CPU scheduler. (STS)

These are called short and long based upon frequency of task. like STS has to do tasks very quickly in ms.

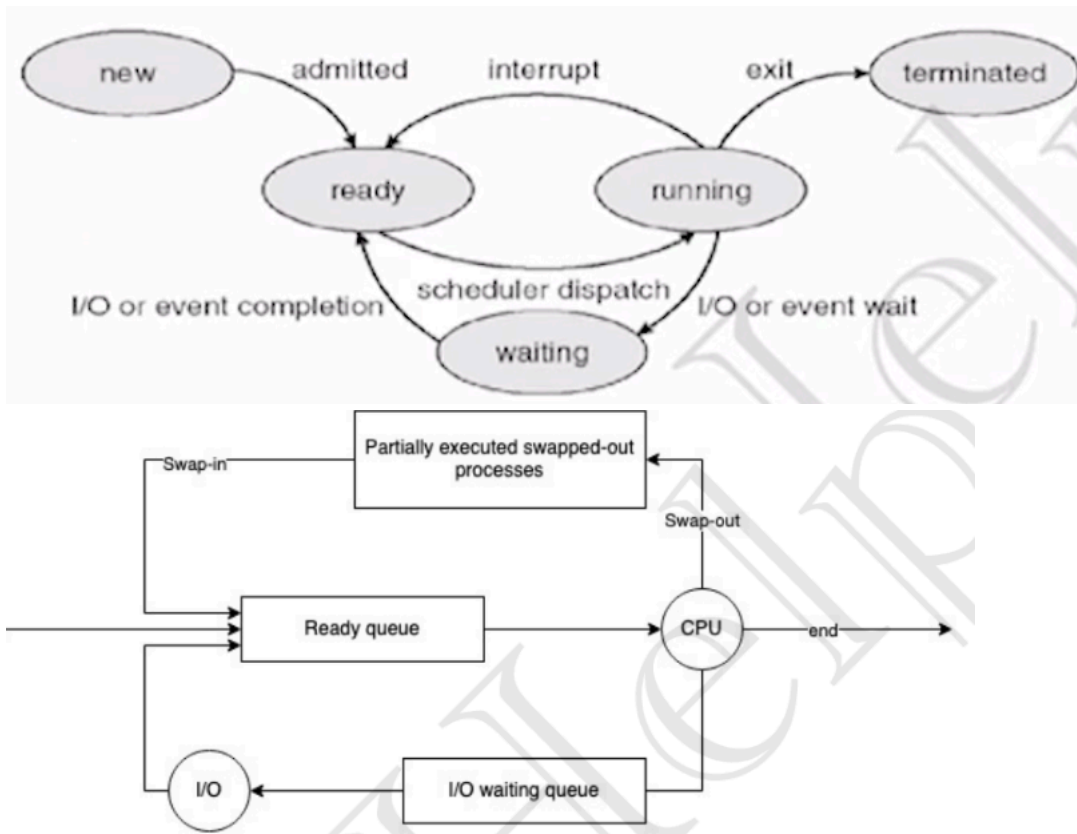
	LTS	STS
Frequency	Low	Very high
Decides	Which new processes enter memory/ready queue	Which ready process gets CPU
Queue	Job queue → Ready queue	Ready queue → CPU
Goal	Control degree of multiprogramming	Maximise CPU utilisation & responsiveness
Speed required	Relatively slower	Must be very fast

Dispatcher:

The short-term scheduler selects a process from the ready queue to execute next, whereas the dispatcher gives the CPU to the selected process by performing the context switch, switching modes if required, and transferring control to the process's next instruction.

Degree of multi programming is the no of jobs in ready queue, which is affected / managed by LTS.

LTS should choose good mix of processes that means: not all IO or not all CPU bound processes. If ready queue has memory intensive tasks, some processes from ready queue get swapped out to secondary memory and later loaded back, this is done using Medium Term Scheduler.



Context Switching in OS:

Context switching is the process of saving the state (context) of the currently running process/thread and loading the saved state of another process/thread so the CPU can switch execution between them.

Done by Kernel and using PCB's for contexts of processes.

Orphan Process:

Using `fork()` command we make child processes of a parent.

the ultimate parent (1st process of OS) process is **init** which has a PID of 1.

When P1 is running and creates P2 using `fork()` and P1 gets exception and terminates, then P1 becomes an orphan process, OS assigns init as P1's parent in this case.

Zombie Process:

Process whose execution is completed but still has entry in the process table. (ps and see hidden using `ps -a`).

Read more on this.

Process Scheduling:

The process of providing CPU to process based on some logics/algorithms.

Types of processes:

1. Preemptive: Leaves CPU when termination, IO or when time quantum expires.
2. Non-preemptive: Leaves CPU only on termination or IO task. (time sharing not possible here)

Starvation: Process is waiting for CPU, more when there are more non-preemptive processes.

Overhead: more when more preemptive processes due to requirement of context switching.

CPU Utilisation: more when more preemptive tasks are present, as no ideal time. (generally, not a hard rule).

Goals of CPU Scheduling:

1. Max CPU utilisation: Simple to understand.
2. Least Turn Around Time (TAT): Time from first time in ready queue to end. (CT - AT).
3. Least Wait Time (WT): Total Time process spends waiting for CPU (TAT - BT).
4. Least Response Time (RT): Time to get CPU for the first time after being in ready queue.
5. Max Throughput: Processes completed per unit time.

Other terms:

Arrival Time: Time of arrival of process in the ready queue for the first time.

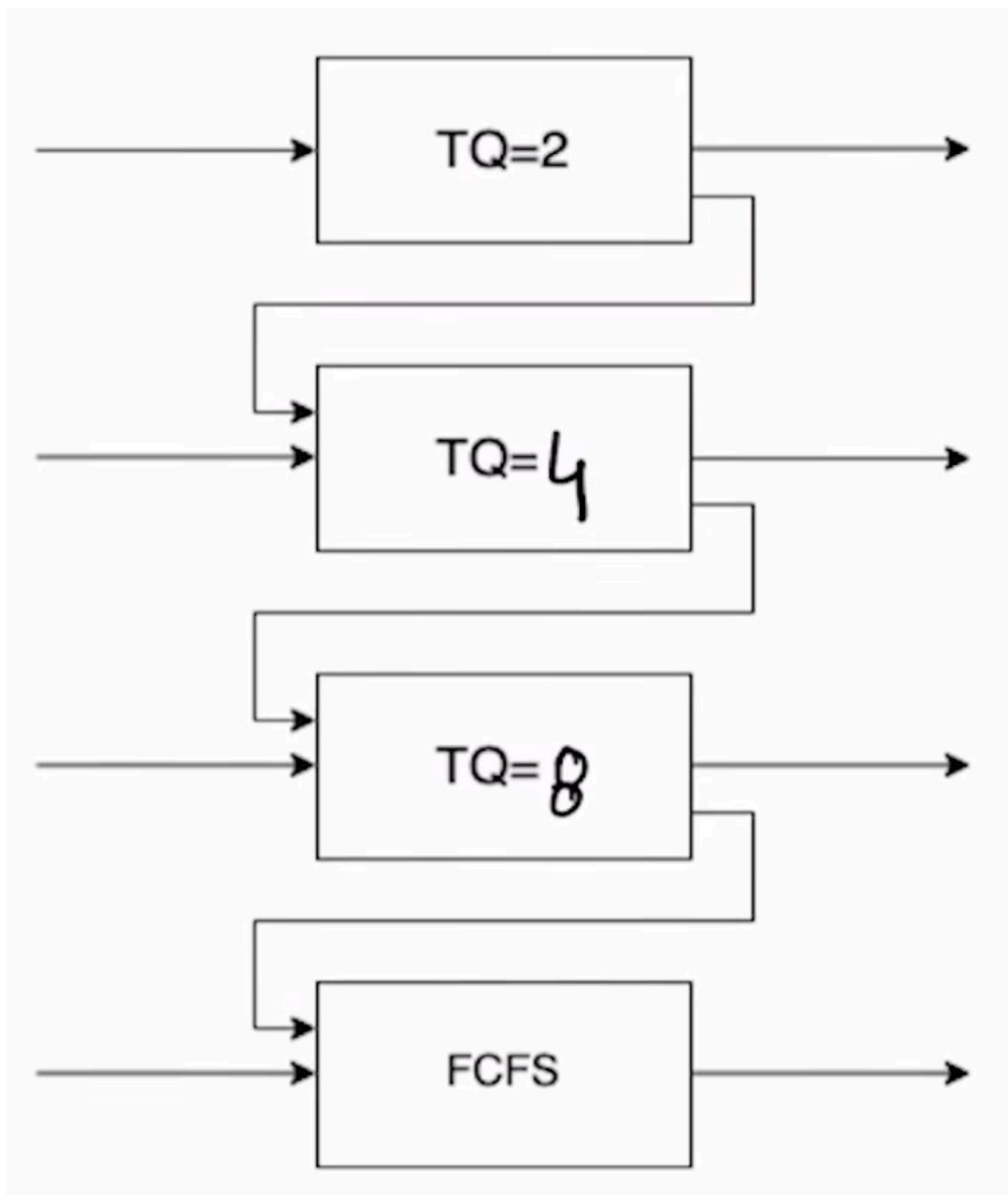
Burst Time: Total time needed by the process for execution.

Completion Time: Time when process gets executed completely.

Convoy Effect: When larger BT processes comes first so smaller BT processes have to wait for long. Average WT increases. occurs in FCFS, non preemptive SJF, both priority scheduling.

SCHEDULING ALGORITHMS:

1. First come First serve (FCFS): Early arrival time gets executed first. Causes **convoy effect**.
2. Shortest Job First (SJF): Least AT, then Least BT will get CPU. non preemptive and preemptive both versions are there. practically not possible as we don't really know BT of processes. preemptive SJF cannot have convoy effect as process arrives and we stop the on going higher BT and carry on with lower BT.
3. Priority Scheduling: Highest priority first, both versions here also. Issue of convoy (indefinite wait time) solved using aging, increasing priorities gradually over time for the processes waiting.
4. Round Robin: AT + TQ is the criteria, idea is preemption + FCFS, designed for time sharing systems, overheads of context switching are high.
5. Multi Level Queue Scheduling: Ready queue is divided into multiple(3) ready queues and based on a process is a system process, Interactive (foreground) process or batch (background) process it is assigned a queue and once assigned it can not be assigned to another queue and each queue has its own scheduling algorithm. And preemptive priority scheduling is used to schedule among different sub-queues, with highest priority of system process queue and lowest for background process queue. Starvation present. Convoy effect present. No inter queue movement one process stays in one queue only based on its type.
6. Multi Level Feedback Queue: Multiple sub queues but processes can change among different queues. higher burst time processes get assigned to lower level of queues and IO and interactive processes move in high level of queue. Aging method is used to prevent starvation. It is a flexible model and is designed in following steps:
 - no of queues
 - scheduling algorithm of each queue
 - how to promote a process
 - how to demote a process
 - how to assign first queue



Final Comparison:

	FCFS	SJF	PSJF	Priority	P-Priority	RR	MLQ	MLFQ
Design	Simple	Complex	Complex	Complex	Complex	Simple	Complex	Complex
Preemption	No	No	Yes	No	Yes	Yes	Yes	Yes
Convoy effect	Yes	Yes	No	Yes	Yes	No	Yes	Yes
Overhead	No	No	Yes	No	Yes	Yes	Yes	Yes

Concurrency:

Concurrency is doing multiple tasks at overlapping time intervals not exact same time. example: CPU context switching, whereas Parallelism is doing multiple tasks at exact same time. example: multiple CPUs.

Concurrency is must when we want to utilise CPU most, as without this CPU will sit idle when one process sits.

Race Conditions in OS:

A problem may occur when multiple processes or threads concurrently or in parallel access and modify some shared data, and final result differs on their execution order, this is called **race condition**.

This part of program from where this shared data is accessed and modified is known as **critical section**.

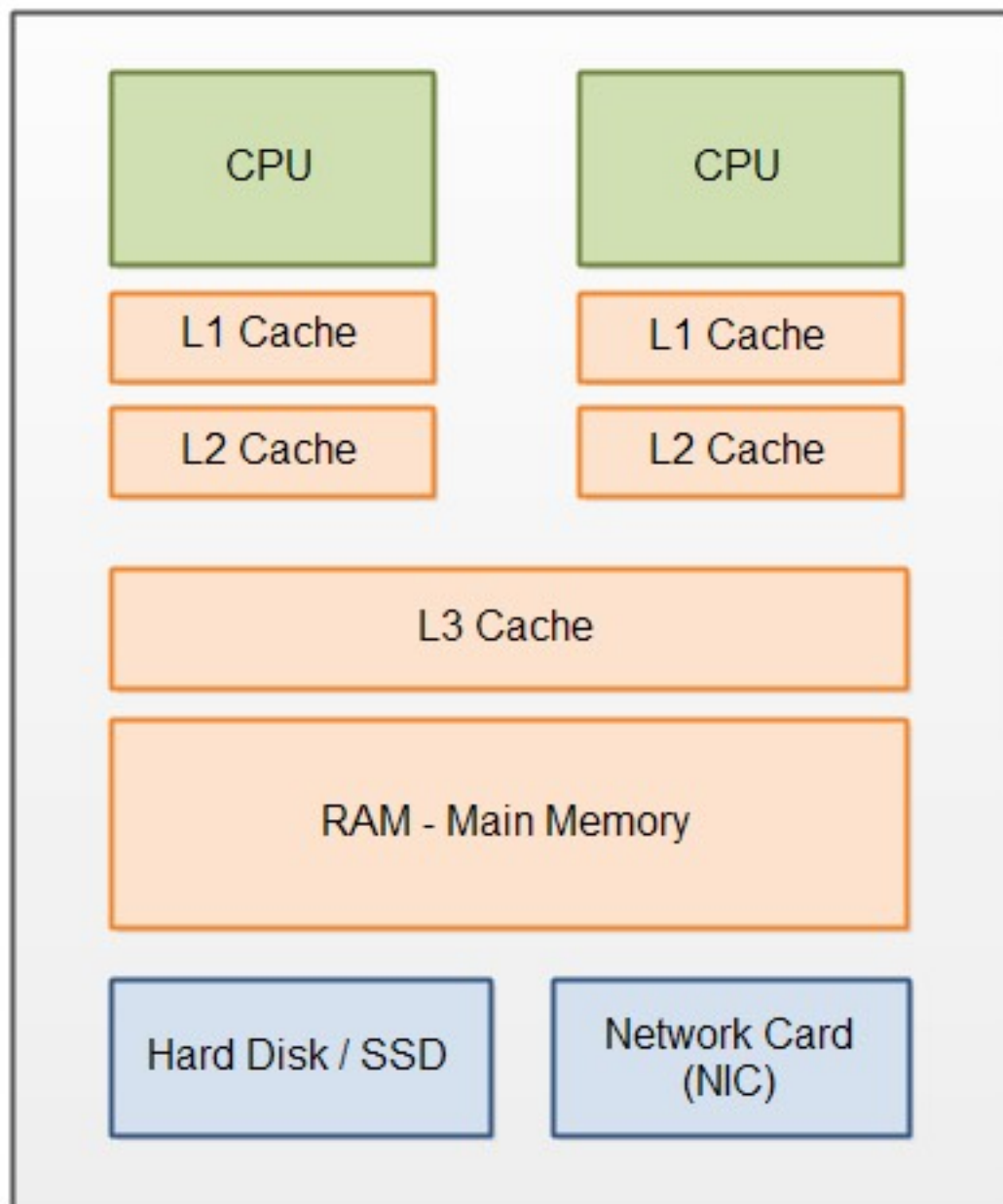
Threads and Concurrency:

When a process has multiple threads running in parallel across different CPU cores, the threads themselves stay in RAM, but the specific data they are actively working on is pulled directly into the **CPU caches and registers**.

At the hardware level, CPU cores absolutely communicate with each other about locks and data changes using a dedicated hardware system called Cache Coherency.

Here is exactly what happens when threads on different cores use shared data and locks:

- **Tracking the State:** The CPU tracks every piece of data in its cache with a "status label." It knows if it is the *only* core looking at that data (**Exclusive**), if other cores have a copy (**Shared**), or if it has modified the data (**Modified**).
- **The Broadcast (Snooping):** When Core 1 wants to acquire a lock or change shared data, it broadcasts an alert over a dedicated internal bus. Core 2 is constantly "snooping" (listening to) this bus.
- **Invalidating Old Data:** The moment Core 1 updates the shared data or claims a lock in its L1 cache, Core 2's hardware instantly marks its own local copy of that data as **Invalid**.
- **Forcing a Reload:** If the thread on Core 2 tries to read that data again, the core sees the "Invalid" tag. It is forced to wait and pull the freshly updated data (or the lock status) from Core 1 or the shared L3 cache/RAM.



Solutions to race condition:

1. Atomic Operation (in cpp): Do the whole critical part and then only context switching is allowed.
2. Mutual Exclusion using Locks: No 2 threads can access at same time.
3. Conditional Variables: Uses locking underneath and has 2 methods - wait and signal. Used when working of 1 thread depends upon task to be done by thread 2. Avoids busy waiting, by acknowledging thread 1 that something has changed so continue executing.
4. Semaphores: Used in case of multiple instances of resources are available, binary and counting semaphore depending upon how many threads we can give access at once.

Conditions for Solutions to work:

1. Mutual Exclusion: No 2 threads can access at same time.

2. *Progress*: Equal opportunity for each thread when there is no one in critical section.
3. *Bounded waiting*: indefinite waiting should not occur.

Can simple flag work? NO

it does bring mutual exclusion but no progress.

so for that there is Peterson's solution. but limited to 2 threads.

Problems of locks:

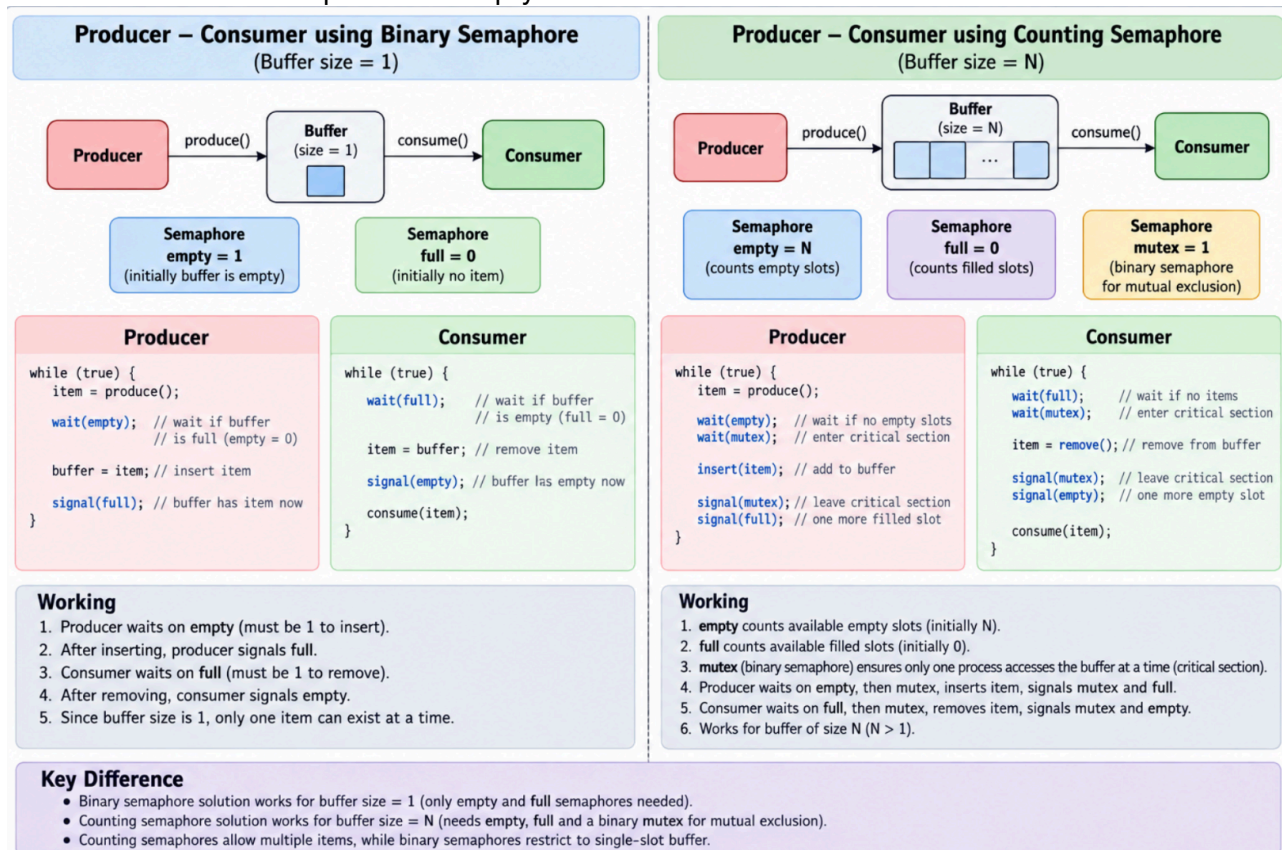
1. *Contention*: indefinite waiting for other threads that were unable to enter.
2. *Deadlocks*: resource already allocated to someone else.
3. *Debugging Issue*: Difficult.
4. *Starvation*: Late arriving high priority job have to wait.

Producer Consumer Problem: (Bounded Buffer Problem)

1 producer thread and 1 consumer thread, the buffer where producer puts anything is a shared resource.

Problems to solve here:

1. Sync producer and consumer threads.
2. Producer must not put in full buffer.
3. Consumer must not pick from empty buffer.



Reader-Writer Problem:

Some read threads and some write threads to a database(shared resource).

if more than 1 reader -> no issue.

if more than 1 writers or 1 writer and other thread are any read or write (dirty reads will occur)-> Semaphore.

1. **Mutex (Binary Semaphore)**: To ensure mutual exclusion when read count is updated, no 2 threads modify this at same time.
2. **Write (Binary Semaphore)**: Common for both reader and writer.
3. **Read Count (integer)**: Maintains count of readers.

Readers – Writers Problem

Multiple readers can read simultaneously, but writers need exclusive access.

1. Reader–Priority Solution

Readers are never blocked (unless a writer is writing).



Shared Variables

```
int readCount = 0;
semaphore mutex = 1; // to protect readCount
semaphore write = 1; // to provide exclusive access to writer
```

Reader Process

```
while (true) {
    wait(mutex);
    readCount++;
    if (readCount == 1)
        wait(write); // first reader locks writer
    signal(mutex);

    // ----- reading section -----
    read();
    // -----

    wait(mutex);
    readCount--;
    if (readCount == 0)
        signal(write); // last reader releases writer
    signal(mutex);
}
```

Writer Process

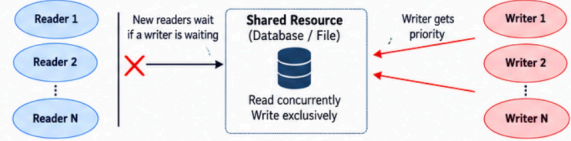
```
while (true) {
    wait(write); // exclusive access
    // ----- writing section -----
    write();
    // -----
    signal(write); // release
}
```

How it Works

1. Multiple readers can read simultaneously.
2. First reader locks the writer (wait(write)).
3. Last reader releases the writer.
4. Writer waits until there are no active readers.
5. Simple, but writers can starve if readers keep coming.

2. Writer–Priority Solution

If a writer is waiting, new readers are blocked.



Shared Variables

```
int readCount = 0;
int writeCount = 0;
semaphore mutex = 1; // to protect readCount and writeCount
semaphore resource = 1; // shared resource (read/write)
semaphore readTry = 1; // blocks readers if writer is waiting
```

Reader Process

```
while (true) {
    wait(readTry); // block if writer waiting
    wait(mutex);
    readCount++;
    if (readCount == 1)
        wait(resource); // first reader locks resource
    signal(mutex);
    signal(readTry); // allow other readers
    // ----- reading section -----
    read();
    // -----

    wait(mutex);
    readCount--;
    if (readCount == 0)
        signal(resource); // last reader releases
    signal(mutex);
}
```

Writer Process

```
while (true) {
    wait(mutex);
    writeCount++;
    if (writeCount == 1)
        wait(readTry); // block new readers
    signal(mutex);

    // ----- writing section -----
    write();
    // -----

    signal(resource);
    wait(mutex);
    writeCount--;
    if (writeCount == 0)
        signal(readTry); // allow readers
    signal(mutex);
}
```

How it Works

1. Writers indicate their intent by incrementing writeCount and blocking new readers (wait(readTry)).
2. Existing readers finish, then writer gets exclusive access.
3. After writing, if no more writers, readTry is released to allow readers.
4. Prevents writer starvation (unlike reader-priority).

Comparison

Aspect	Reader–Priority	Writer–Priority
Readers	Never blocked (unless writing)	Blocked if a writer is waiting
Writers	May starve	Not starved
Fairness	Favors readers	Favors writers
Use case	Read-heavy workloads	Balanced / write-sensitive workloads

Key Takeaways

- Reader–Priority allows more concurrency for readers but can starve writers.
- Writer–Priority ensures writers get access, preventing writer starvation.
- Both use semaphores to synchronize access to a shared resource.
- Can be implemented using mutexes and condition variables as well.

Real-world Examples

- Database systems (multiple reads, exclusive writes)
- File systems
- Configuration management
- Shared caches

Dinning Philosopher Problem:

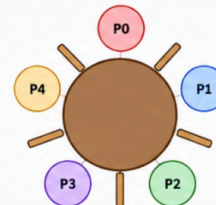
Dining Philosophers Problem

N philosophers sit around a circular table with N chopsticks. Each philosopher alternates between thinking and eating. To eat, a philosopher must pick up both the left and right chopstick. The challenge is to avoid deadlock, starvation and ensure mutual exclusion.

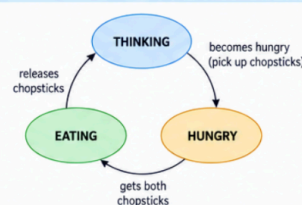
Problem Setup

- N philosophers sit around a round table.
- There is one chopstick between each pair.
- A philosopher can either THINK or EAT.
- To eat, they must pick up both the left and right chopstick.
- Only one philosopher can use a chopstick at a time (mutual exclusion).
- Goal: design a solution that avoids deadlock, starvation and ensures fair access.

5 Philosophers Example (N = 5)

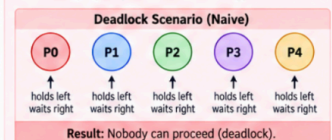


States of a Philosopher



Issues in Naive Approach

- **Deadlock:** all philosophers pick up their left chopstick and wait forever for the right one.
- **Starvation:** a philosopher may never get a chance to eat.
- Need a proper synchronisation strategy.



1. Naive Solution (Leads to Deadlock)

Each philosopher picks up left, then right chopstick.

```
while (true) {
    think();
    wait(chopstick[left]); // pick left
    wait(chopstick[right]); // pick right
    eat();

    signal(chopstick[right]);
    signal(chopstick[left]);
}
```

Problem

- If all philosophers pick up their left chopstick simultaneously, none can get the right one.
- Leads to deadlock.

2. Resource Ordering Solution

Pick up chopsticks in a fixed global order (to avoid circular wait).

```
while (true) {
    think();
    int first = min(left, right);
    int second = max(left, right);
    wait(chopstick[first]);
    wait(chopstick[second]);
    eat();
    signal(chopstick[second]);
    signal(chopstick[first]);
}
```

Why it works

- All philosophers follow the same ordering (lower-numbered chopstick first).
- Breaks the circular wait condition.
- Deadlock is prevented.
- Starvation is still possible (but rare).

3. Waiter Solution (Semaphore)

Use a counting semaphore (N - 1) to allow at most N - 1 philosophers to try picking chopsticks.

```
semaphore room = N - 1; // allows N-1
semaphore chopstick[N] = {1}; // each chopstick

while (true) {
    think();
    wait(room);
    wait(chopstick[left]); // enter room
    wait(chopstick[right]);
    eat();
    signal(chopstick[right]);
    signal(chopstick[left]);
    signal(room); // leave room
}
```

Why it works

- At most N - 1 philosophers can sit (try to eat) at a time.
- Ensures at least one philosopher can always get both chopsticks.
- Prevents deadlock.
- Simple and effective.

4. Monitor Solution (Using States + Condition Variables)

Use a monitor with states and condition variables to manage access fairly.

```
enum State { THINKING, HUNGRY, EATING };
State state[N];
condition self[N];

void take_forks(int i) {
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING)
        wait(self[i]);
}

void put_forks(int i) {
    state[i] = THINKING;
    test((i + 1) % N); // check right neighbour
    wait((i - 1 + N) % N); // check left neighbour
}

void test(int i) {
    if (state[i] == HUNGRY &&
        state[(i + 1) % N] != EATING &&
        state[(i - 1 + N) % N] != EATING) {
        state[i] = EATING;
        signal(self[i]);
    }
}
```

Key Idea

- Each philosopher has a state: THINKING, HUNGRY, or EATING.
- A philosopher can eat only if both neighbours are not eating.
- Uses condition variables for waiting and signalling.
- Prevents deadlock and can be made starvation-free with fair scheduling (e.g., FIFO).

Comparison

Approach	Deadlock	Starvation	Complexity	Notes
Naive	Yes	Yes	Low	Simple but incorrect
Resource Ordering	No	Possible	Low	Breaks circular wait
Waiter (Semaphore)	No	Possible	Low	At most N-1 try to eat
Monitor (Condition Vars)	No	No (with fairness)	Medium	Most practical solution

Key Takeaways

- Deadlock happens due to circular wait (all waiting for each other).
- Break at least one of the four Coffman conditions (e.g., circular wait or hold-and-wait).
- Dining Philosophers teaches synchronization, resource allocation and concurrency control.
- Real-world applications: database locks, resource sharing, thread synchronization, and avoiding deadlocks in distributed systems.

Coffman Conditions for Deadlock

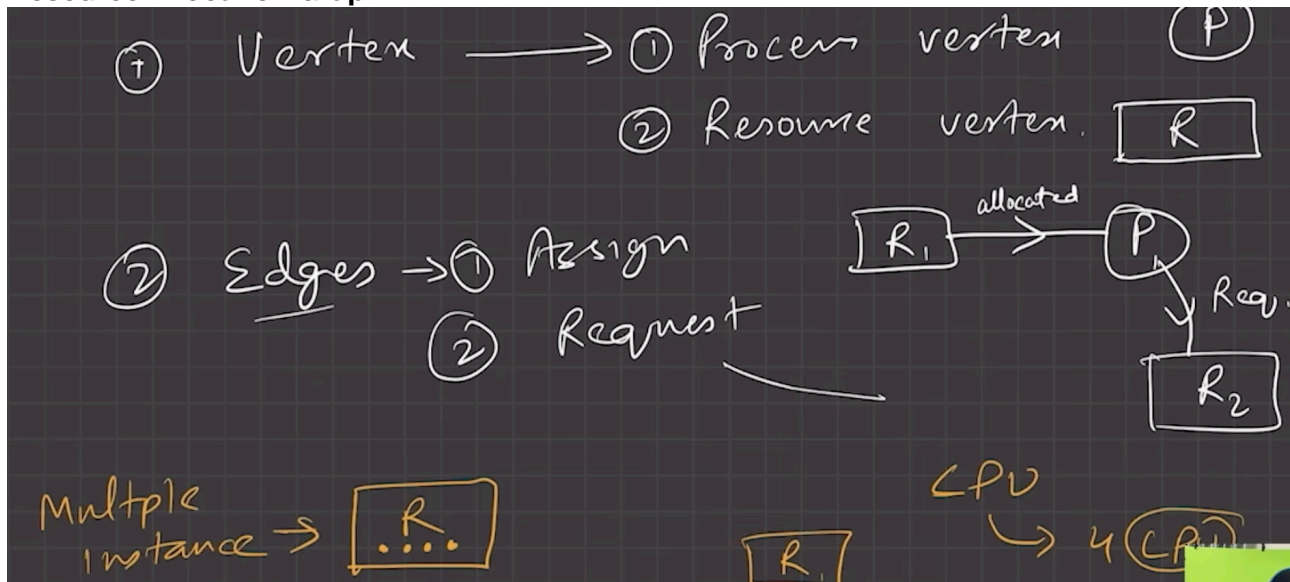
1. **Mutual exclusion** – only one can use a resource at a time.
 2. **Hold and wait** – holding one resource while waiting for another.
 3. **No preemption** – resources cannot be forcibly taken.
 4. **Circular wait** – a circular chain of waiting processes.
- Break at least one to prevent deadlock.**

Deadlocks:

Necessary conditions for deadlock:

1. *Mutual Exclusion*: Any resource R1 can only be used by 1 process/thread at a time. (resource POV)
2. *Hold and Wait*: until work is not done resource must be locked by P1. (Process POV)
3. *No preemption*: not released till execution of allocated process is completed.
4. *Circular Wait*: allocation and demand is in circular fashion.

Resource Allocation Graph:



cycle -> **may be** dead lock.

Handle deadlock:

1. Prevent or avoid
2. Allow to go in deadlock, detect, solve
3. Ostrich Algorithm - ignorance

Prevention Techniques: (do not let to go in deadlock)

Make any one of the 4 conditions of deadlock false.

1. Sharable resources like read only files can be used for violating mutual exclusion.
2. Allocate ALL resources before execution starts and allow process to req only when it has 0 resources.
3. release resources and wait for them to be free and lock all at same time when available.

Avoidance techniques: (deadlock occurred, now what?)

1. Bankers Algorithm: We need to schedule processes in such a way that they have to end up in safe states.

Deadlock Detection: (we let go system in deadlock and then recover it afterwards)

1. *Single instance of each resource (wait for graph)* - remove all the resource edges and if cycle occurs its DL.
2. *Multiple instance*: Bankers algorithm

Recovery from deadlock:

1. *Process Termination*: abort process of low priority.
2. *Resource Preemption*: free resource causing deadlock.

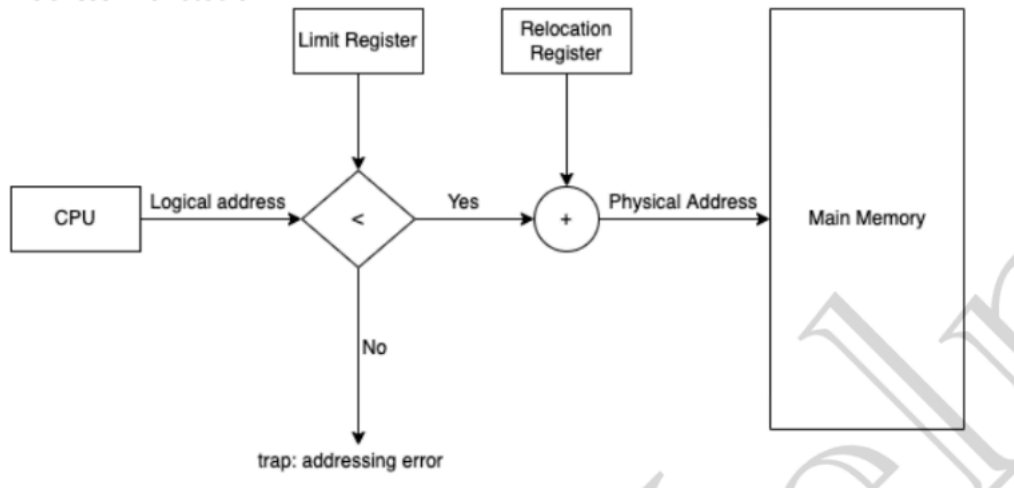
Memory Management in OS:

OS allocates a base in RAM (physical memory space) and stores offset for each process.

Physical Address	Logical Address
address loaded in the memory register of physical memory (main memory)	address used by any process
Generated by MMU	Generated by CPU
user can't access this	user can access this
$R + 0$ to $R + \text{max}$ (R is base)	Virtual Address (0 to max)

MMU:

h. Address Translation



Types of allocation:

1. **Contiguous:** Complete process gets memory at one time. **Fixed Partitioning** (blocks divided have equal size or variable size - suppose there are 4MB partitions and 5 processes come 3MB each, i will not be able to give 4MB to last process although its available - this is called **external fragmentation**, And the wastage to 1MB in each allocation is termed as **internal fragmentation** Also if a process of 5MB comes then this can't be loaded. Also degree of multiprogramming is very less in this scenario - equal to number of partitions available). Dynamic Partitioning (Variable size blocks decided at time when process comes, this still has external fragmentation issue.)
2. **Non - contiguous:** Linked List (free list) is used to store the free space available. Solves external fragmentation.

Both are contiguous allocation techniques:

Feature	Fixed Partitioning	Dynamic Partitioning
Partition creation	At system startup	Created when process arrives
Partition size	Fixed	Variable , based on process size
Number of partitions	Fixed	Changes dynamically
Main wastage	Internal fragmentation	External fragmentation
Process fitting	Process must fit inside an existing partition	Allocate a block approximately equal to process size
Flexibility	Low	High
Memory utilisation	Lower	Generally better

Compaction	Usually not needed	May be required
Example	1 GB RAM → four 250 MB partitions	Process asks 180 MB → allocate ~180 MB

Defragmentation/Compaction in OS

Defragmentation means reorganising memory so that scattered free spaces are combined into one larger contiguous free block.

CPU does de-fragmentation (abstract function) to minimise probability of external fragmentation. Base address changes (overhead) and offset remains same. Free Space is combined together.

How to assign process the blocks of free space?

After these techniques are applied our allocation becomes contiguous only.

1. First Fit: First capable node of free list.
2. Best Fit: Smallest capable segment to fit the process.
3. Next Fit: Iterate from where ever the counter was left.
4. Worst Fit: Largest hole to accommodate.

Paging:

A method to do non-contiguous allocation taken in action by MMU.

Page is basically the fixed size division of a process's logical space address.

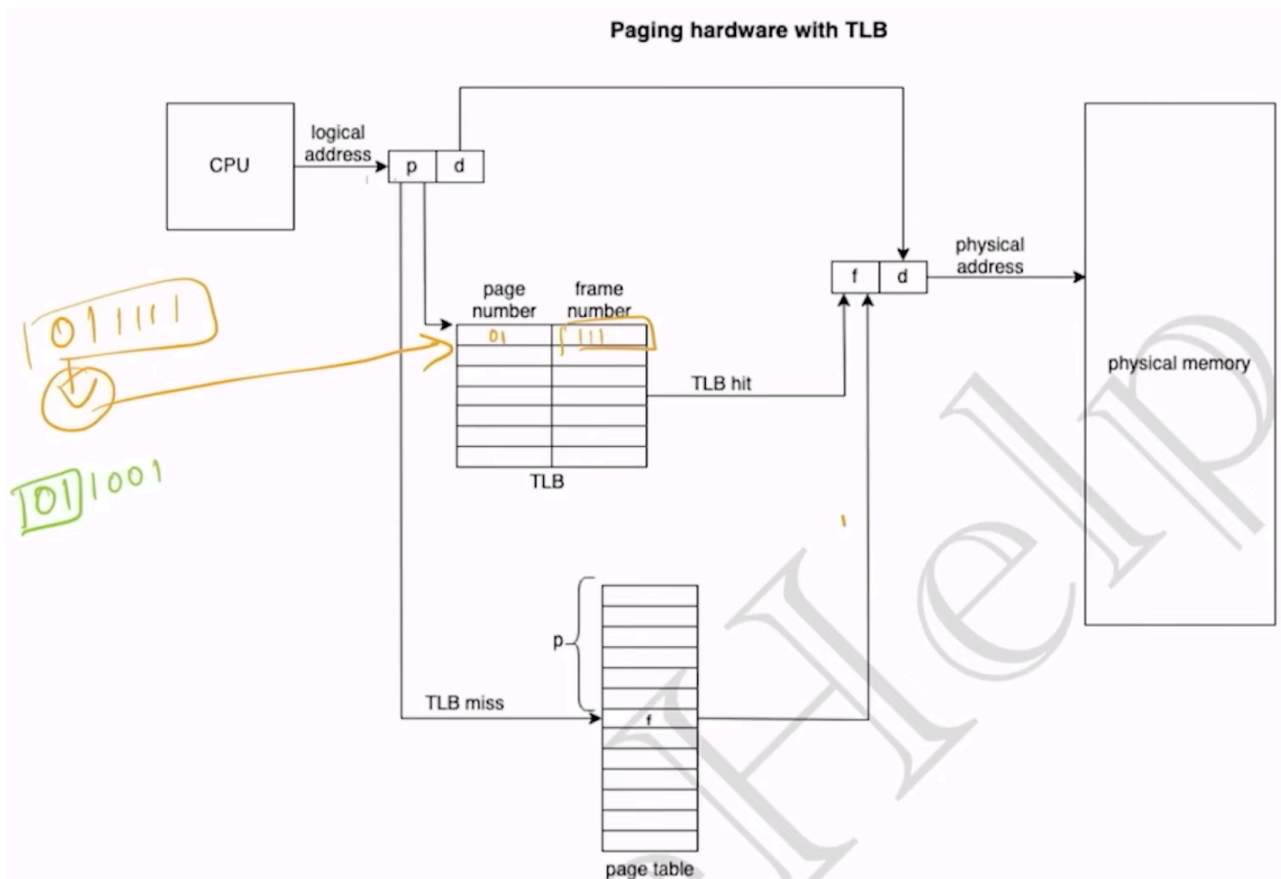
This fixed size partition is also done in RAM to fit page's fixed division in it, which is called **frame**.

Each process has its own **page table** (stores mapping of page no → frame no) and is stored in PCB of process.

PTBR (Page table base register) points to page table of current process. Used during context switching.

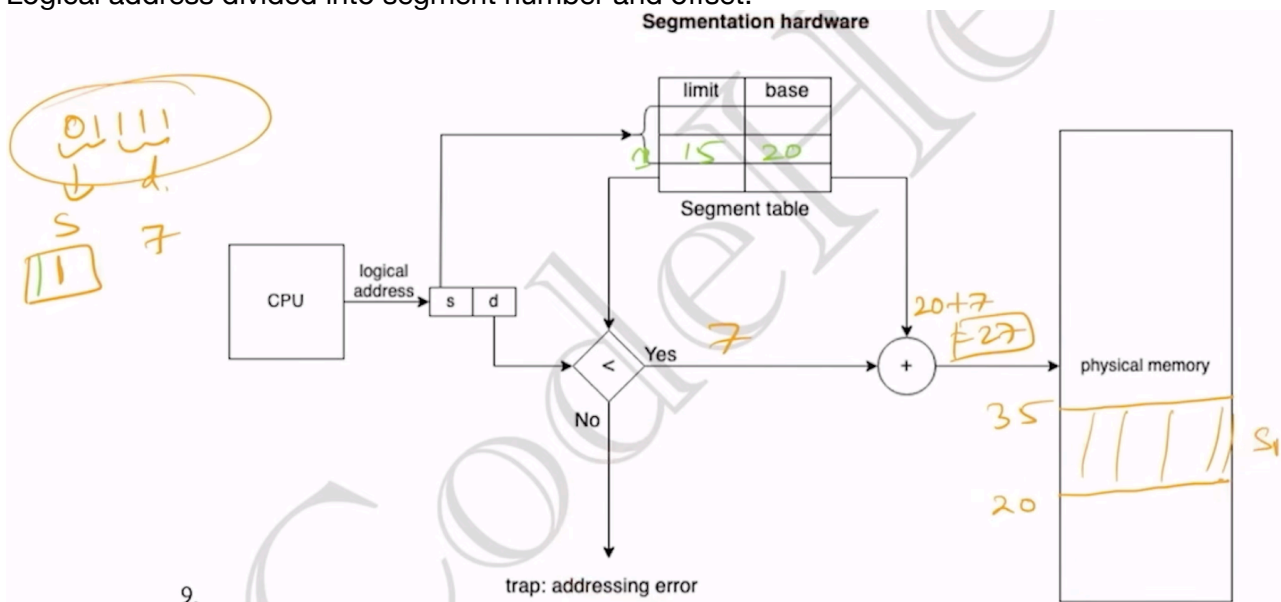
There are a lot of memory references in this so paging is slow, so we have to do caching.

Translation look-ahead buffer: A cache table used to find the frame number corresponding to the page number. It is common for multiple processes as it stores PID (ASID - address space identifier), Page number and Frame number. So we do not need to flush it when process switches. Therefore, Page table for each process is different but not TLB.



Segmentation:

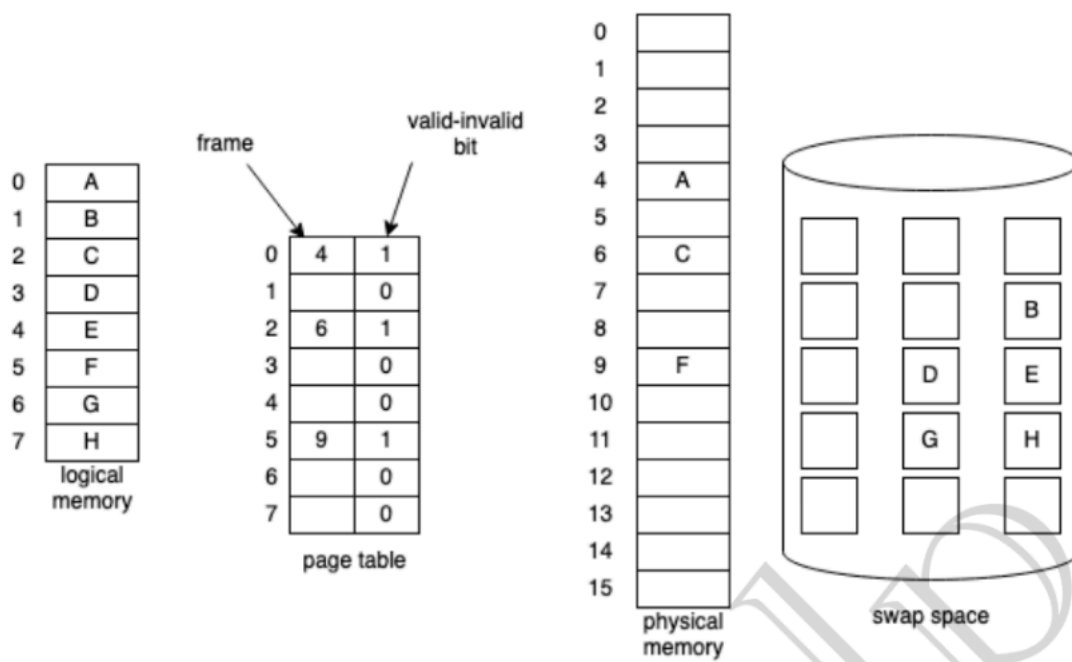
Variable partitioning of logical address space (so every partition has different base address and offset, here called limit). So here MMU maintains a segment table rather than a PT. Logical address divided into segment number and offset.



Virtual Memory: a technique to execute processes that are not in physical memory. A part of secondary memory is treated as main memory called **swap space (done by Pager)**. Only the needed pages of a program are loaded in the main memory.

So degree of multi programming increases (but thrashing may also occur) as we can run more processes than total RAM. This virtual memory management is called **Page Demanding**.

Page table when some pages are not in memory



e.

Page Replacement Algorithm:

Page faults occur when a page is demanded and it is not currently in the physical memory and needs to be loaded from the virtual memory.

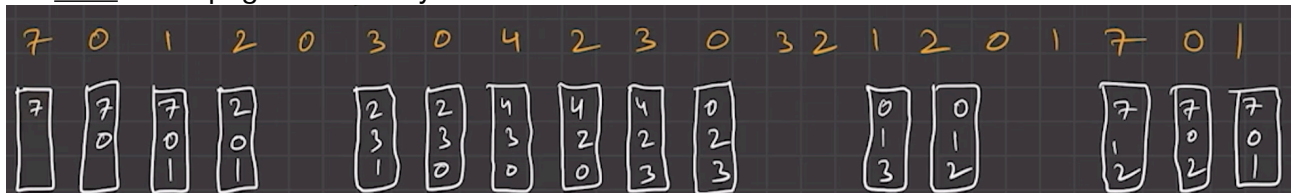
Who brings page in? PAGER.

Who brings complete processes in? Swapper.

How to decide which page should be swapped? Page Replacement Algorithm.

Page Replacement Algorithms:

1. FIFO: lot of page faults. Easy to do.



2. Optimal Page Replacement: Impossible to implement. We remove page from frame which will be needed late.
3. Least Recently Used: simple.
4. Least Frequently Used: simple.
5. Most Frequently used: simple.

Belady Anomaly:

As no of frames increase the total page faults decrease.
But in FIFO it does not happen, that's anomaly.

Thrashing:

When CPU is more busy serving page faults rather than executing processes. occurs usually in high degree of multiprogramming.

Precautions:

1. Working set model: Allocate all locality pages of a process to RAM at once, but if number of frames are less than size of current locality process is bound to thrash.

2. Page Fault Frequency: Set a lower and upper bound on number of pages from each process to occupy the frames.

Disk Scheduling:

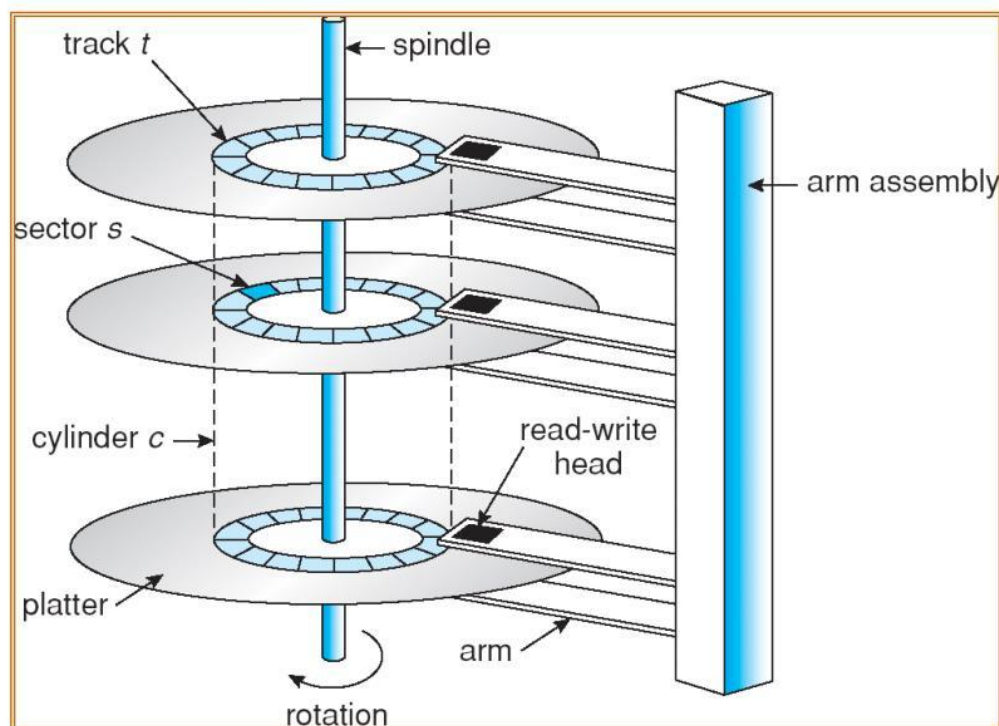
1. What a disk physically/logically is
2. How an OS accesses data on disk
3. How the OS decides which disk request to serve next — disk scheduling

A **disk** is a secondary storage device used to store data **persistently**. eg - HDD (Hard Disk Drive), SSD (Solid State Drive)

RAM	Disk
Volatile	Non-volatile
Loses data on power off	Retains data
Very fast	Much slower
Expensive per GB	Cheaper per GB
Directly used for active programs	Used for persistent storage

We generally talk about HDD as they are accessed by mechanical movement of a disk head.

Physical Disk Structure



Platter - Circular magnetic disks where data is stored, each has 2 surfaces (top n bottom), there may be multiple of these in a hard disk.

Spindle - This rotates all platters, higher RPM generally means lower rotational latency.

Read/Write head - each platter surface has one of these, moves across surface using actuator arm.

Actuator Arm - moves head between tracks, major delay - seek time.

Track - A circular path on a platter surface. (many concentric on 1 surface)

Sectors - A track is divided into sectors. (have physical storage blocks)

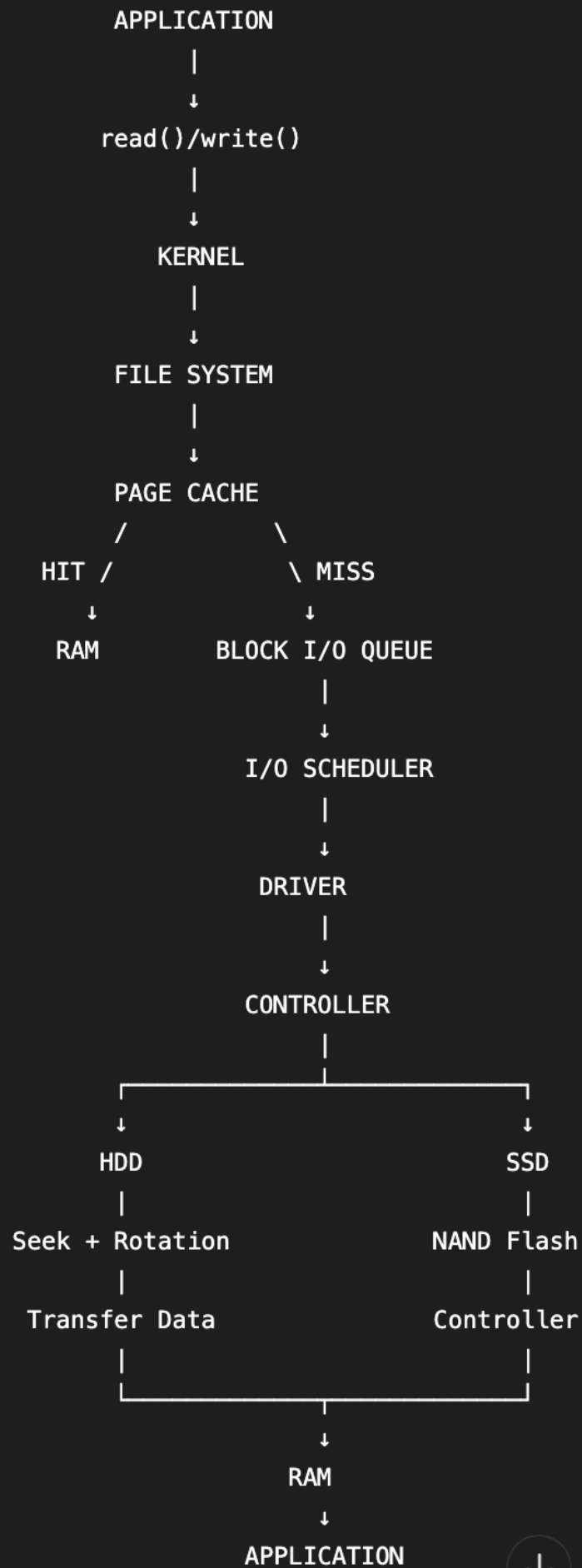
Disk Access Time = Seek Time + Rotational Latency + Transfer Time + Controller/queue overhead

Time	Meaning
Seek Time	Move head to target track
Rotational Latency	Wait for sector to rotate under head
Transfer Time	Actually transfer data
Disk Access Time	Overall time to access data
Response Time	Request submission → response
Queueing Time	Time waiting in request queue

Now there are a few algorithms of disk scheduling by which we judge which request should be served next:

1. FCFS - first come first serve, simple, fair, no starvation, bad average seek time, lot of movements.
2. SSTF - shortest seek time first, choose req closest to head, can cause starvation for far away track.
3. SCAN - Elevator algo, go towards predefined direction, goes till extreme end if requests are remaining in queue, starvation and bad average in case of dynamic adding of requests.
4. C-SCAN - circular scan, scans in one direction only and starts from 0 again, more uniform waiting time.
5. LOOK - moves in both direction only till last request.
6. C-LOOK - this moves only in one direction and only till last request.

Algorithm	Main Idea	Starvation?	Head Movement
FCFS	Arrival order	No	High
SSTF	Closest request	Yes	Low
SCAN	Elevator	No	Moderate
C-SCAN	Circular elevator	No	Moderate
LOOK	SCAN without unnecessary end travel	No	Lower
C-LOOK	C-SCAN without unnecessary end travel	No	Lower



Other Terms:

Virtual memory

Cache vs Buffer

Page Cache

Page Faults

How does data access take place?

DMA - Direct memory Access

56. What Happens During a File Read?

Interviewers may ask:

"What happens when I read a file?"

A strong answer:

```
Application
  ↓
read() system call
  ↓
Kernel
  ↓
File system checks file metadata
  ↓
Check page cache
  ↓
If cache hit → return data
  ↓
If cache miss
  ↓
Generate block I/O request
  ↓
Block layer queues request
  ↓
Device driver/controller
  ↓
Storage device
  ↓
Data transferred to memory
  ↓
I/O completion
  ↓
Process receives data
```

Gaps that could trip you up:

1. **No numerical practice** — your notes are all conceptual. Interviewers frequently give you a burst-time table and ask you to compute average waiting time / turnaround time for FCFS/SJF/RR, or trace a page-replacement sequence (FIFO/LRU) for hits/faults. Definitions alone won't get you through that.
2. **File systems missing entirely** — inode structure, directory structures, file allocation methods (contiguous/linked/indexed), and file system implementation are a common OS interview bucket and aren't in these notes at all.
3. **Semaphores are named but not implemented** — you list "Semaphores:" with nothing under it. Be ready to explain `wait()/signal()`, binary vs. counting semaphore, and how they differ from mutexes (ownership, use case) — this is a very common gotcha question.
4. **User-level vs. kernel-level threads** — you cover thread vs. process well but not this distinction, which comes up when threading is discussed.
5. **Belady's Anomaly** — a classic follow-up when FIFO page replacement comes up ("can more frames ever cause more faults?").
6. **Dining philosophers** — named as a heading with no solution written. If it's in your notes as a topic, interviewers may expect you to at least sketch a resource-ordering or semaphore-based fix.
7. **Monitors** as a higher-level synchronization construct (alongside semaphores) sometimes comes up in language-specific contexts (Java `synchronized`, etc.).